RAW DATA TO CLEAN DATA CONVERSION USING PYTHON EDA

In [1]:
```
import pandas as pd
```

In [2]:
```
pd.__version__
```

Out[2]:
```
'2.3.3'
```

In [3]:
```
# pip install --upgrade openpyxl
```

In [4]:
```
emp = pd.read_excel(r'D:\Rawdata.xlsx')
```

In [5]:
```
emp
```

Out[5]:

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Datascience#$	34 years	Mumbai	5^00#0	2+
1	Teddy^	Testing	45' yr	Bangalore	10%%000	<3
2	Uma#r	Dataanalyst^^#	NaN	NaN	1$5%000	4> yrs
3	Jane	Ana^^lytics	NaN	Hyderbad	2000^0	NaN
4	Uttam*	Statistics	67-yr	NaN	30000-	5+ year
5	Kim	NLP	55yr	Delhi	6000^$0	10+

In [6]:
```
id(emp)
```

Out[6]:
```
2425701659184
```

In [7]:
```
emp.columns
```

Out[7]:
```
Index(['Name', 'Domain', 'Age', 'Location', 'Salary', 'Exp'], dtype='object')
```

In [8]:
```
emp.shape
```

Out[8]:
```
(6, 6)
```

In [9]:
```
emp.head()
```

Out[9]:

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Datascience#$	34 years	Mumbai	5^00#0	2+
1	Teddy^	Testing	45' yr	Bangalore	10%%000	<3
2	Uma#r	Dataanalyst^^#	NaN	NaN	1$5%000	4> yrs
3	Jane	Ana^^lytics	NaN	Hyderbad	2000^0	NaN
4	Uttam*	Statistics	67-yr	NaN	30000-	5+ year

In [10]:

```
emp.tail()
```

Out[10]:

	Name	Domain	Age	Location	Salary	Exp
1	Teddy^	Testing	45' yr	Bangalore	10%%000	<3
2	Uma#r	Dataanalyst^^#	NaN	NaN	1$5%000	4> yrs
3	Jane	Ana^^lytics	NaN	Hyderbad	2000^0	NaN
4	Uttam*	Statistics	67-yr	NaN	30000-	5+ year
5	Kim	NLP	55yr	Delhi	6000^$0	10+

In [11]:

```
emp.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6 entries, 0 to 5
Data columns (total 6 columns):
 #   Column    Non-Null Count  Dtype 
---  ------    --------------  ----- 
 0   Name      6 non-null      object
 1   Domain    6 non-null      object
 2   Age       4 non-null      object
 3   Location  4 non-null      object
 4   Salary    6 non-null      object
 5   Exp       5 non-null      object
dtypes: object(6)
memory usage: 420.0+ bytes
```

In [12]:

```
emp
```

Out[12]:

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Datascience#$	34 years	Mumbai	5^00#0	2+
1	Teddy^	Testing	45' yr	Bangalore	10%%000	<3
2	Uma#r	Dataanalyst^^#	NaN	NaN	1$5%000	4> yrs
3	Jane	Ana^^lytics	NaN	Hyderbad	2000^0	NaN
4	Uttam*	Statistics	67-yr	NaN	30000-	5+ year
5	Kim	NLP	55yr	Delhi	6000^$0	10+

In [13]:

```
emp['Domain']
```

Out[13]:

```
0      Datascience#$
1            Testing
2     Dataanalyst^^#
3        Ana^^lytics
4         Statistics
5                NLP
Name: Domain, dtype: object
```

In [14]:

```
emp.isnull()
```

Out[14]:

	Name	Domain	Age	Location	Salary	Exp
0	False	False	False	False	False	False
1	False	False	False	False	False	False
2	False	False	True	True	False	False
3	False	False	True	False	False	True
4	False	False	False	True	False	False
5	False	False	False	False	False	False

In [15]:

```
emp.isna()
```

Out[15]:

	Name	Domain	Age	Location	Salary	Exp
0	False	False	False	False	False	False
1	False	False	False	False	False	False
2	False	False	True	True	False	False
3	False	False	True	False	False	True
4	False	False	False	True	False	False
5	False	False	False	False	False	False

In [16]:
```
emp.isnull().sum()
```

Out[16]:
```
Name        0
Domain      0
Age         2
Location    2
Salary      0
Exp         1
dtype: int64
```

In [17]:
```
emp.columns
```

Out[17]:
```
Index(['Name', 'Domain', 'Age', 'Location', 'Salary', 'Exp'], dtype='object')
```

In [18]:
```
emp
```

Out[18]:

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Datascience#$	34 years	Mumbai	5^00#0	2+
1	Teddy^	Testing	45' yr	Bangalore	10%%000	<3
2	Uma#r	Dataanalyst^^#	NaN	NaN	1$5%000	4> yrs
3	Jane	Ana^^lytics	NaN	Hyderbad	2000^0	NaN
4	Uttam*	Statistics	67-yr	NaN	30000-	5+ year
5	Kim	NLP	55yr	Delhi	6000^$0	10+

In [19]:
```
emp.dtypes
```

Out[19]:
```
Name        object
Domain      object
Age         object
Location    object
Salary      object
Exp         object
dtype: object
```

DATA CLEANING OR DATA CLEANSING

In [20]:
```
emp['Name']
```

Out[20]:
```
0      Mike
1    Teddy^
2     Uma#r
3      Jane
4    Uttam*
5       Kim
Name: Name, dtype: object
```

In [21]:
```
emp['Name'] = emp['Name'].str.replace(r'\W', '', regex = True)  #non word charac
emp['Name']
```

Out[21]:
```
0     Mike
1    Teddy
2     Umar
3     Jane
4    Uttam
5      Kim
Name: Name, dtype: object
```

In [22]:
```
emp['Domain'] = emp['Name'].str.replace(r'\W+','',regex=True)
emp['Domain']
```

Out[22]:
```
0     Mike
1    Teddy
2     Umar
3     Jane
4    Uttam
5      Kim
Name: Domain, dtype: object
```

In [23]:
```
emp['Age'] = emp['Age'].str.extract('(\d+)')
emp['Age']
```

Out[23]:
```
0     34
1     45
2    NaN
3    NaN
4     67
5     55
Name: Age, dtype: object
```

In [27]:
```
emp['Location'] = emp['Location'].str.replace(r'\W','')
emp['Location']
```

Out[27]:
```
0       Mumbai
1    Bangalore
2          NaN
3     Hyderbad
4          NaN
5        Delhi
Name: Location, dtype: object
```

In [24]:
```
emp['Salary'] = emp['Salary'].str.replace(r'\W','',regex=True)
emp['Salary']
```

Out[24]:
```
0     5000
1    10000
2    15000
3    20000
4    30000
5    60000
Name: Salary, dtype: object
```

In [25]:
```
emp['Age'] = emp['Age'].str.replace(r'\W+','',regex=True)
emp['Age']
```

Out[25]:
```
0     34
1     45
2    NaN
3    NaN
4     67
5     55
Name: Age, dtype: object
```

In [26]:
```
emp
```

Out[26]:

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Mike	34	Mumbai	5000	2+
1	Teddy	Teddy	45	Bangalore	10000	<3
2	Umar	Umar	NaN	NaN	15000	4> yrs
3	Jane	Jane	NaN	Hyderbad	20000	NaN
4	Uttam	Uttam	67	NaN	30000	5+ year
5	Kim	Kim	55	Delhi	60000	10+

In [28]:
```
emp['Exp'] = emp['Exp'].str.replace(r'\W','',regex=True)
emp['Exp']
```

Out[28]:
```
0        2
1        3
2     4yrs
3      NaN
4    5year
5       10
Name: Exp, dtype: object
```

In [29]:
```
emp['Exp'] = emp['Exp'].str.extract(r'(\d+)')
# to clean txt use extract then we get fully clend exp
emp['Exp']
```

Out[29]:
```
0      2
1      3
2      4
3    NaN
4      5
5     10
Name: Exp, dtype: object
```

In [30]:
```
emp    # look at the data it is half cleaned
```

Out[30]:

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Mike	34	Mumbai	5000	2
1	Teddy	Teddy	45	Bangalore	10000	3
2	Umar	Umar	NaN	NaN	15000	4
3	Jane	Jane	NaN	Hyderbad	20000	NaN
4	Uttam	Uttam	67	NaN	30000	5
5	Kim	Kim	55	Delhi	60000	10

In [31]:

```
clean_data = emp.copy() # we are copying the emp data to clean_data
clean_data
```

Out[31]:

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Mike	34	Mumbai	5000	2
1	Teddy	Teddy	45	Bangalore	10000	3
2	Umar	Umar	NaN	NaN	15000	4
3	Jane	Jane	NaN	Hyderbad	20000	NaN
4	Uttam	Uttam	67	NaN	30000	5
5	Kim	Kim	55	Delhi	60000	10

MISSING VALUE TREATEMENT

In [44]:

```
clean_data
```

Out[44]:

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Mike	34	Mumbai	5000	2
1	Teddy	Teddy	45	Bangalore	10000	3
2	Umar	Umar	50.25	NaN	15000	4
3	Jane	Jane	50.25	Hyderbad	20000	4.8
4	Uttam	Uttam	67	NaN	30000	5
5	Kim	Kim	55	Delhi	60000	10

In [45]:

```
clean_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6 entries, 0 to 5
Data columns (total 6 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   Name      6 non-null      object
 1   Domain    6 non-null      object
 2   Age       6 non-null      object
 3   Location  4 non-null      object
 4   Salary    6 non-null      object
 5   Exp       6 non-null      object
dtypes: object(6)
memory usage: 420.0+ bytes
```

In [32]:
```
import numpy as np
```

clean_data

In [46]:
```
clean_data.head(1)
```

Out[46]:

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Mike	34	Mumbai	5000	2

In [33]:
```
clean_data['Age'] # here 2,3 has no value we will fill by mean,median,mode for n
```

Out[33]:
```
0     34
1     45
2    NaN
3    NaN
4     67
5     55
Name: Age, dtype: object
```

In [34]:
```
clean_data ['Age']=clean_data['Age'].fillna(np.mean(pd.to_numeric(clean_data['Age']
clean_data ['Age']
```

Out[34]:
```
0       34
1       45
2    50.25
3    50.25
4       67
5       55
Name: Age, dtype: object
```

In [35]:
```
clean_data['Exp'] # here 3 has no value we will fill by mean,median,mode for num
```

Out[35]:
```
0      2
1      3
2      4
3    NaN
4      5
5     10
Name: Exp, dtype: object
```

In [37]:
```
clean_data ['Exp']= clean_data['Exp'].fillna(np.mean(pd.to_numeric(clean_data['Exp'
clean_data['Exp']
```

Out[37]:
```
0      2
1      3
2      4
3    4.8
4      5
5     10
Name: Exp, dtype: object
```

In [38]:
```
clean_data
```

Out[38]:

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Mike	34	Mumbai	5000	2
1	Teddy	Teddy	45	Bangalore	10000	3
2	Umar	Umar	50.25	NaN	15000	4
3	Jane	Jane	50.25	Hyderbad	20000	4.8
4	Uttam	Uttam	67	NaN	30000	5
5	Kim	Kim	55	Delhi	60000	10

In [39]:
```
# now for catogorical data we use mode/ KNN
```

In [40]:
```
clean_data['Location'] # here we have catogorical data or text here 2, 4 hasno v
```

Out[40]:
```
0       Mumbai
1    Bangalore
2          NaN
3     Hyderbad
4          NaN
5        Delhi
Name: Location, dtype: object
```

In [47]:
```
clean_data['Location'] =clean_data['Location'].fillna(clean_data['Location'].mode()
clean_data['Location']
```

Out[47]:
```
0       Mumbai
1    Bangalore
2    Bangalore
3     Hyderbad
4    Bangalore
5        Delhi
Name: Location, dtype: object
```

In [48]:
```
clean_data
```

Out[48]:

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Mike	34	Mumbai	5000	2
1	Teddy	Teddy	45	Bangalore	10000	3
2	Umar	Umar	50.25	Bangalore	15000	4
3	Jane	Jane	50.25	Hyderbad	20000	4.8
4	Uttam	Uttam	67	Bangalore	30000	5
5	Kim	Kim	55	Delhi	60000	10

In [49]:
```
clean_data['Age'] = clean_data['Age'].astype(int)
```

In [50]:
```
clean_data['Salary'] = clean_data['Salary'].astype(int)
```

In [51]:
```
clean_data['Exp'] = clean_data['Exp'].astype(int)
```

In [52]:
```
clean_data
```

Out[52]:

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Mike	34	Mumbai	5000	2
1	Teddy	Teddy	45	Bangalore	10000	3
2	Umar	Umar	50	Bangalore	15000	4
3	Jane	Jane	50	Hyderbad	20000	4
4	Uttam	Uttam	67	Bangalore	30000	5
5	Kim	Kim	55	Delhi	60000	10

In [53]:
```
clean_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6 entries, 0 to 5
Data columns (total 6 columns):
 #   Column    Non-Null Count  Dtype 
---  ------    --------------  ----- 
 0   Name      6 non-null      object
 1   Domain    6 non-null      object
 2   Age       6 non-null      int64 
 3   Location  6 non-null      object
 4   Salary    6 non-null      int64 
 5   Exp       6 non-null      int64 
dtypes: int64(3), object(3)
memory usage: 420.0+ bytes
```

In [54]:
```
clean_data['Name'] = clean_data['Name'].astype('category')
clean_data['Domain'] = clean_data['Domain'].astype('category')
```

```
clean_data['Location'] = clean_data['Location'].astype('category')
clean_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6 entries, 0 to 5
Data columns (total 6 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   Name      6 non-null      category
 1   Domain    6 non-null      category
 2   Age       6 non-null      int64
 3   Location  6 non-null      category
 4   Salary    6 non-null      int64
 5   Exp       6 non-null      int64
dtypes: category(3), int64(3)
memory usage: 938.0 bytes
```

In [55]:
```
clean_data
```

Out[55]:

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Mike	34	Mumbai	5000	2
1	Teddy	Teddy	45	Bangalore	10000	3
2	Umar	Umar	50	Bangalore	15000	4
3	Jane	Jane	50	Hyderbad	20000	4
4	Uttam	Uttam	67	Bangalore	30000	5
5	Kim	Kim	55	Delhi	60000	10

In [56]:
```
clean_data.to_csv('clean_data.csv')
```

In [57]:
```
import os
os.getcwd()
```

Out[57]: 'C:\\Users\\3star\\codfile'

In [58]:
```
clean_data.columns
```

Out[58]: Index(['Name', 'Domain', 'Age', 'Location', 'Salary', 'Exp'], dtype='object')

In [59]:
```
import matplotlib.pyplot as plt # visualization
import seaborn as sns # Advanced visualization
```

In [60]:
```
import warnings
warnings.filterwarnings('ignore')
```

In [61]:
```
clean_data
```

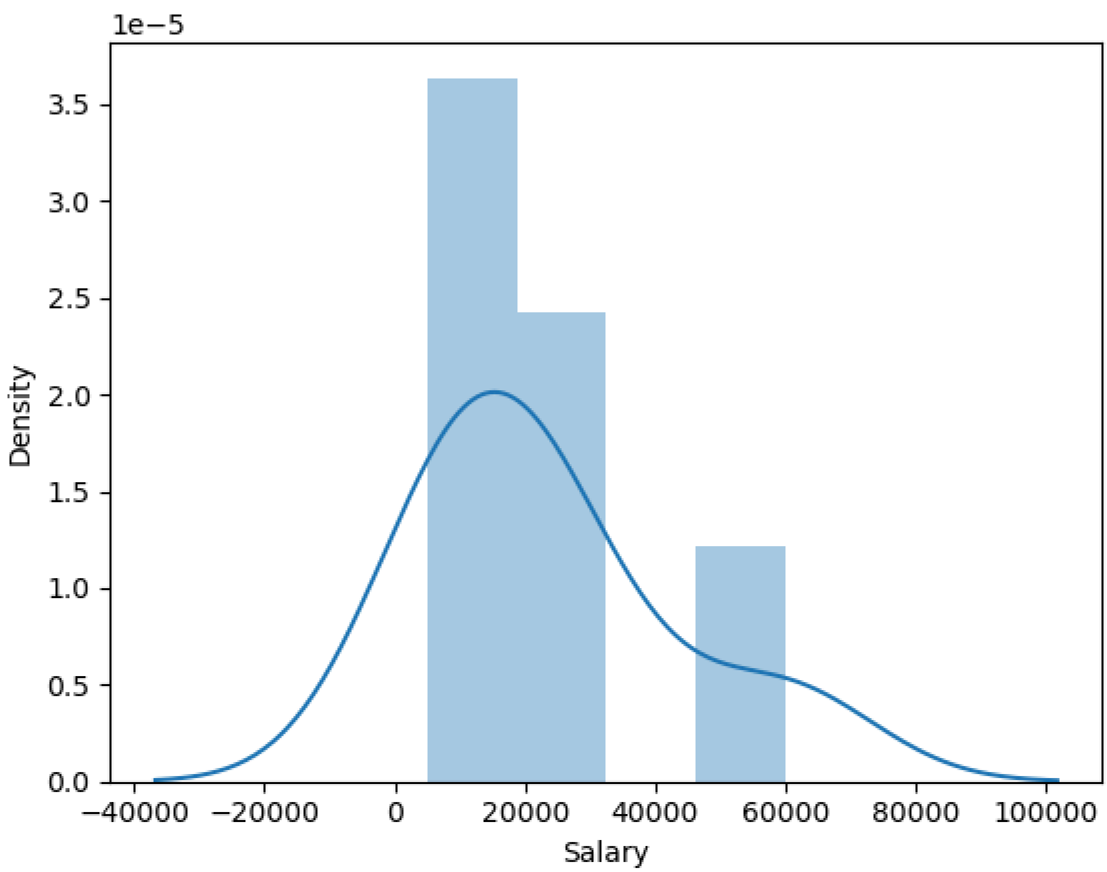
Out[61]:

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Mike	34	Mumbai	5000	2
1	Teddy	Teddy	45	Bangalore	10000	3
2	Umar	Umar	50	Bangalore	15000	4
3	Jane	Jane	50	Hyderbad	20000	4
4	Uttam	Uttam	67	Bangalore	30000	5
5	Kim	Kim	55	Delhi	60000	10

In [62]:
```
clean_data['Salary']
```

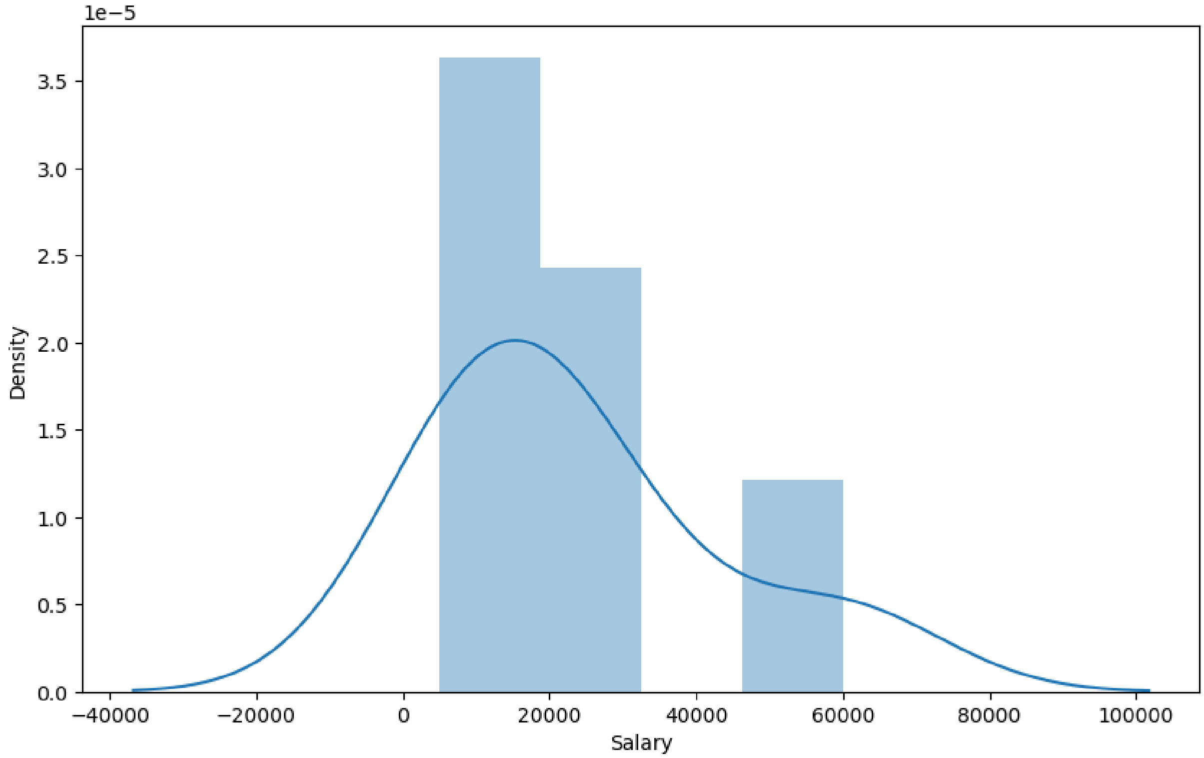
Out[62]:
```
0     5000
1    10000
2    15000
3    20000
4    30000
5    60000
Name: Salary, dtype: int64
```

In [63]:
```
vis1 = sns.distplot(clean_data['Salary'])
```

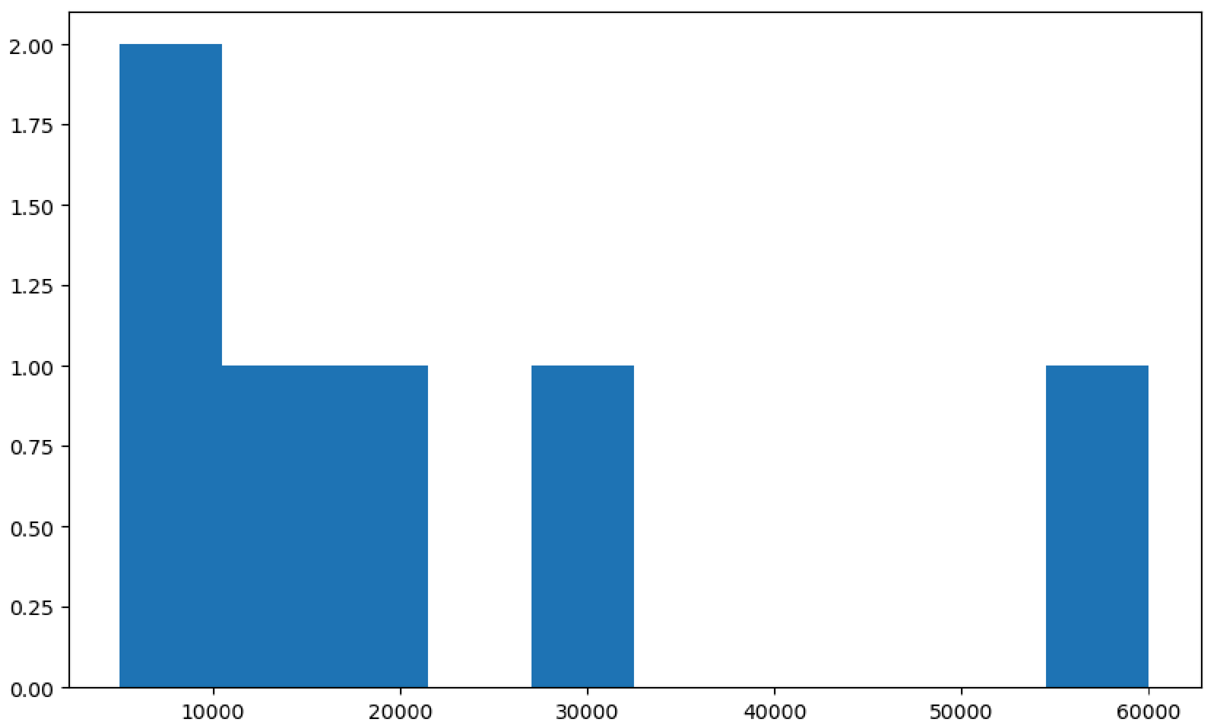


In [64]:
```
plt.rcParams['figure.figsize'] = 10,6
```

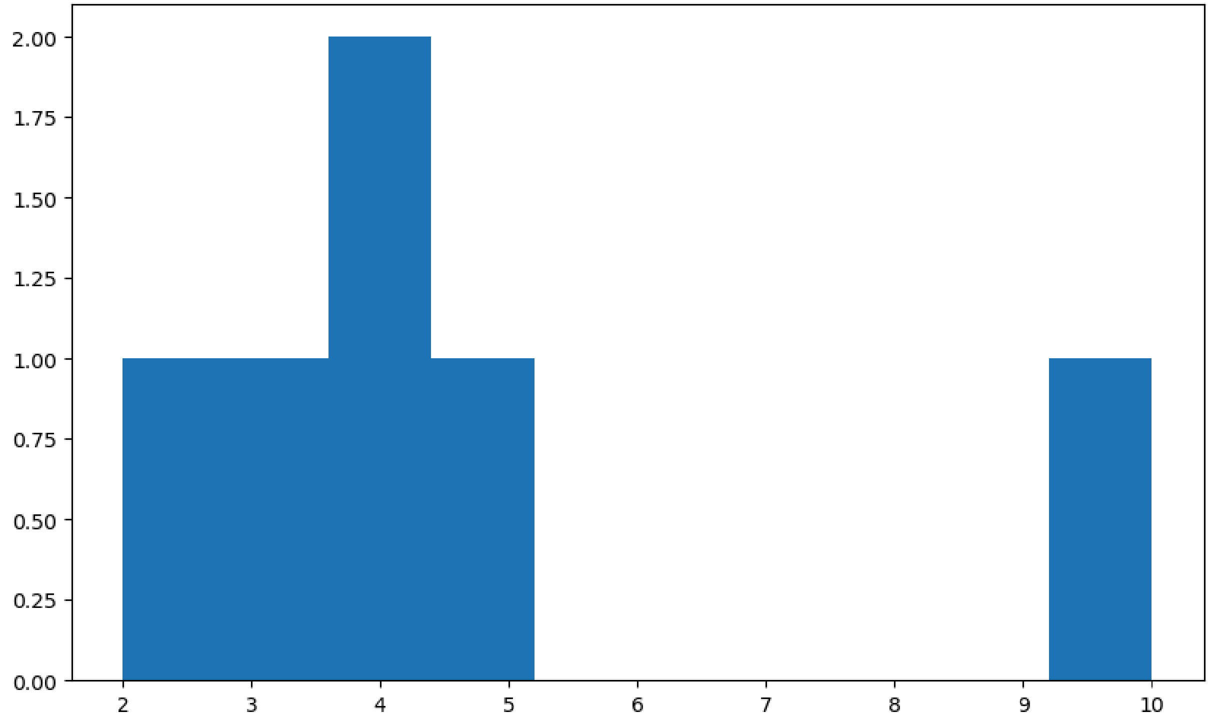
In [65]:
```
vis1 = sns.distplot(clean_data['Salary'])
```



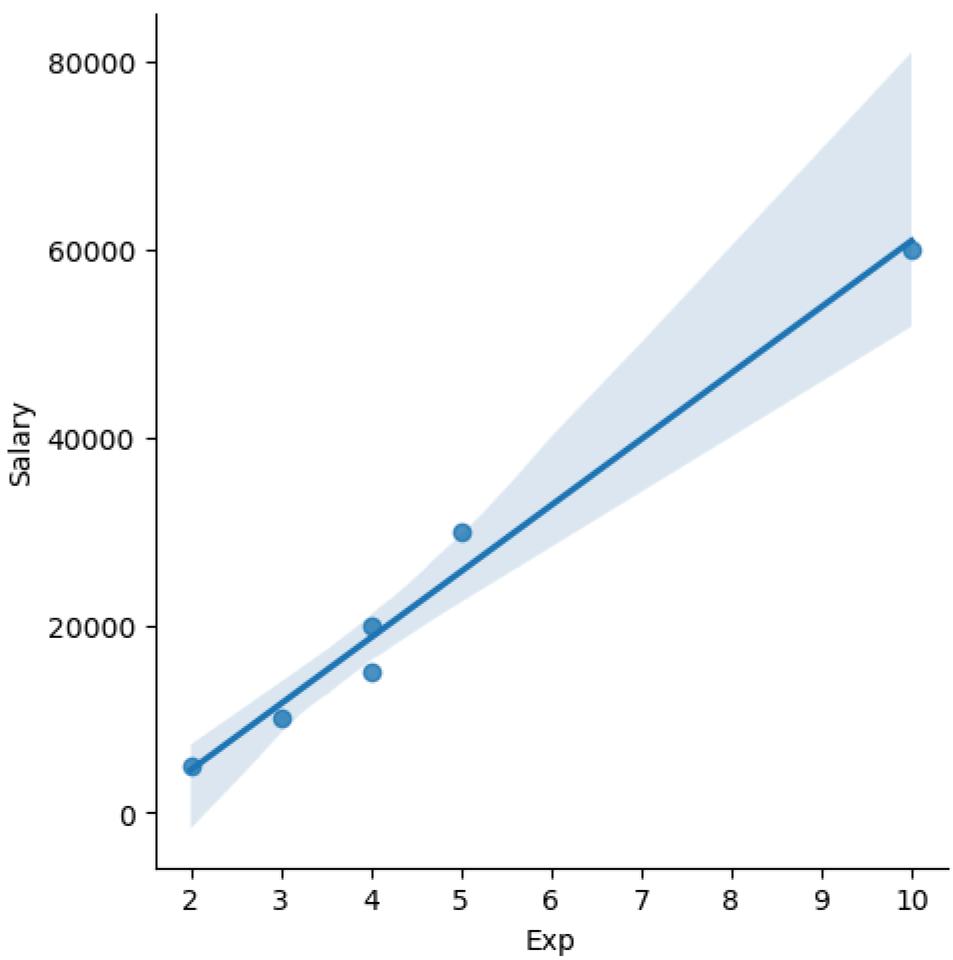
In [66]:
```
vis2 = plt.hist(clean_data['Salary'])
```



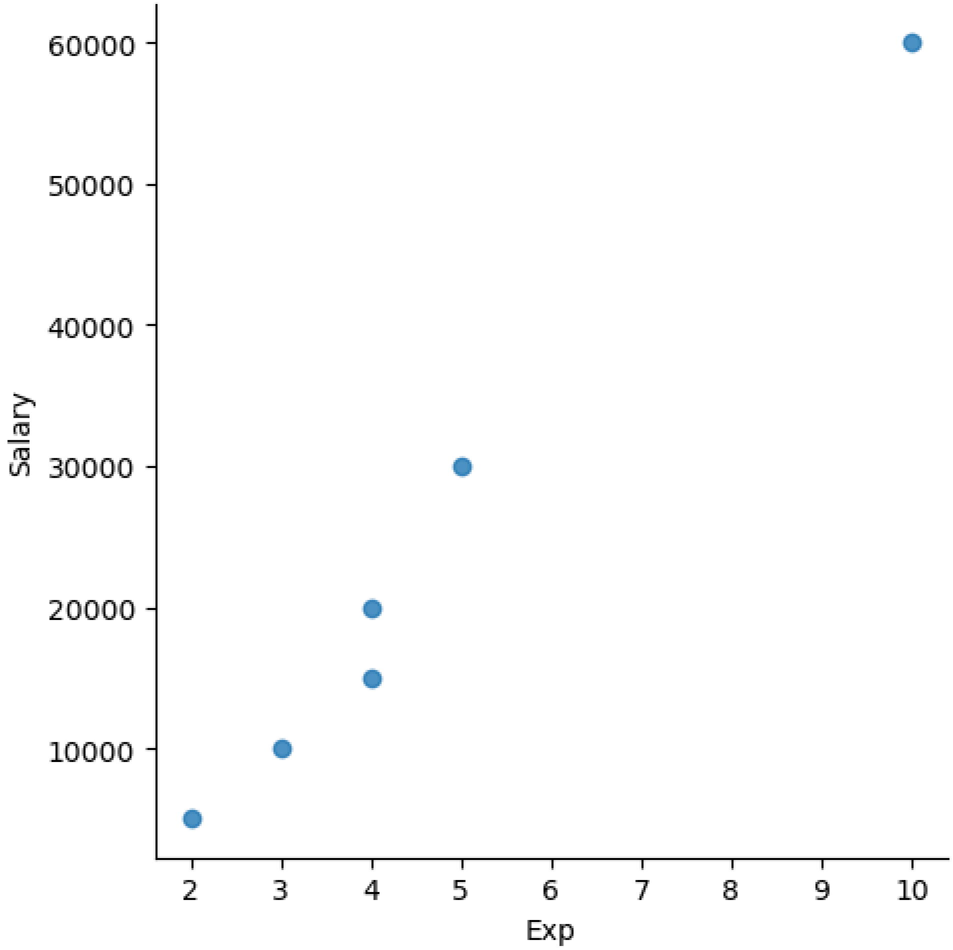
In [67]:
```
vis3 = plt.hist(clean_data['Exp'])
```



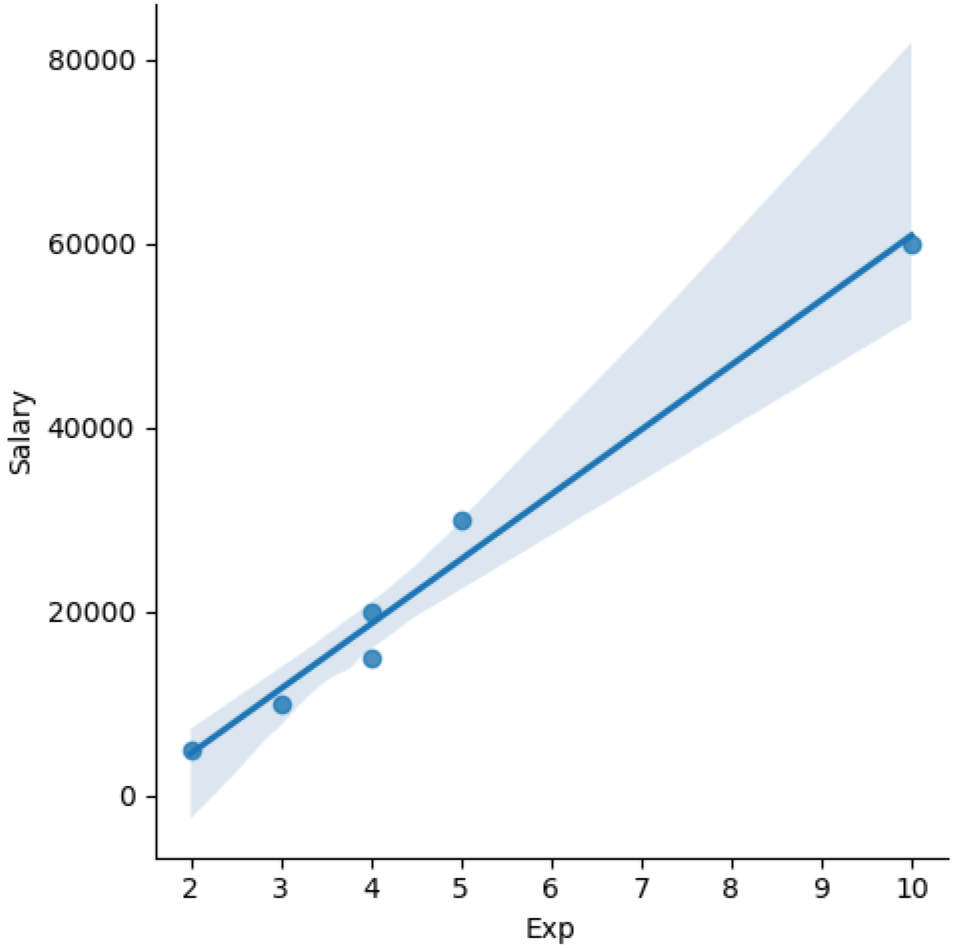
In [68]:
```
vis4 = sns.lmplot(data=clean_data,x = 'Exp', y='Salary')
```



In [69]:
```
vis5 = sns.lmplot(data=clean_data,x = 'Exp', y='Salary', fit_reg = False)
```



In [70]:
```
vis6 = sns.lmplot(data=clean_data,x = 'Exp', y='Salary', fit_reg = True)
```



In [71]:
```
clean_data
```

Out[71]:

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Mike	34	Mumbai	5000	2
1	Teddy	Teddy	45	Bangalore	10000	3
2	Umar	Umar	50	Bangalore	15000	4
3	Jane	Jane	50	Hyderbad	20000	4
4	Uttam	Uttam	67	Bangalore	30000	5
5	Kim	Kim	55	Delhi	60000	10

In [72]:
```
clean_data[:]
```

Out[72]:

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Mike	34	Mumbai	5000	2
1	Teddy	Teddy	45	Bangalore	10000	3
2	Umar	Umar	50	Bangalore	15000	4
3	Jane	Jane	50	Hyderbad	20000	4
4	Uttam	Uttam	67	Bangalore	30000	5
5	Kim	Kim	55	Delhi	60000	10

In [73]:
```
clean_data[:2]
```
Out[73]:

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Mike	34	Mumbai	5000	2
1	Teddy	Teddy	45	Bangalore	10000	3

In [74]:
```
clean_data[2:]
```
Out[74]:

	Name	Domain	Age	Location	Salary	Exp
2	Umar	Umar	50	Bangalore	15000	4
3	Jane	Jane	50	Hyderbad	20000	4
4	Uttam	Uttam	67	Bangalore	30000	5
5	Kim	Kim	55	Delhi	60000	10

In [75]:
```
clean_data[:]
```
Out[75]:

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Mike	34	Mumbai	5000	2
1	Teddy	Teddy	45	Bangalore	10000	3
2	Umar	Umar	50	Bangalore	15000	4
3	Jane	Jane	50	Hyderbad	20000	4
4	Uttam	Uttam	67	Bangalore	30000	5
5	Kim	Kim	55	Delhi	60000	10

In [76]:
```
clean_data[0:1]
```

Out[76]:

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Mike	34	Mumbai	5000	2

In [77]:

```
clean_data[0,3]
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
File D:\Users\3star\Lib\site-packages\pandas\core\indexes\base.py:3812, in Index.get
_loc(self, key)
   3811 try:
-> 3812     return self._engine.get_loc(casted_key)
   3813 except KeyError as err:

File pandas/_libs/index.pyx:167, in pandas._libs.index.IndexEngine.get_loc()

File pandas/_libs/index.pyx:196, in pandas._libs.index.IndexEngine.get_loc()

File pandas/_libs/hashtable_class_helper.pxi:7088, in pandas._libs.hashtable.PyObjec
tHashTable.get_item()

File pandas/_libs/hashtable_class_helper.pxi:7096, in pandas._libs.hashtable.PyObjec
tHashTable.get_item()

KeyError: (0, 3)

The above exception was the direct cause of the following exception:

KeyError                                  Traceback (most recent call last)
Cell In[77], line 1
----> 1 clean_data[0,3]

File D:\Users\3star\Lib\site-packages\pandas\core\frame.py:4113, in DataFrame.__geti
tem__(self, key)
   4111 if self.columns.nlevels > 1:
   4112     return self._getitem_multilevel(key)
-> 4113 indexer = self.columns.get_loc(key)
   4114 if is_integer(indexer):
   4115     indexer = [indexer]

File D:\Users\3star\Lib\site-packages\pandas\core\indexes\base.py:3819, in Index.get
_loc(self, key)
   3814     if isinstance(casted_key, slice) or (
   3815         isinstance(casted_key, abc.Iterable)
   3816         and any(isinstance(x, slice) for x in casted_key)
   3817     ):
   3818         raise InvalidIndexError(key)
-> 3819     raise KeyError(key) from err
   3820 except TypeError:
   3821     # If we have a listlike key, _check_indexing_error will raise
   3822     #  InvalidIndexError. Otherwise we fall through and re-raise
   3823     #  the TypeError.
   3824     self._check_indexing_error(key)

KeyError: (0, 3)
```

In [78]:

```
clean_data
```

Out[78]:

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Mike	34	Mumbai	5000	2
1	Teddy	Teddy	45	Bangalore	10000	3
2	Umar	Umar	50	Bangalore	15000	4
3	Jane	Jane	50	Hyderbad	20000	4
4	Uttam	Uttam	67	Bangalore	30000	5
5	Kim	Kim	55	Delhi	60000	10

In [79]:
```
x_iv = clean_data.drop(['Salary'],axis=1)
```

In [80]:
```
clean_data
```

Out[80]:

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Mike	34	Mumbai	5000	2
1	Teddy	Teddy	45	Bangalore	10000	3
2	Umar	Umar	50	Bangalore	15000	4
3	Jane	Jane	50	Hyderbad	20000	4
4	Uttam	Uttam	67	Bangalore	30000	5
5	Kim	Kim	55	Delhi	60000	10

In [81]:
```
x_iv
```

Out[81]:

	Name	Domain	Age	Location	Exp
0	Mike	Mike	34	Mumbai	2
1	Teddy	Teddy	45	Bangalore	3
2	Umar	Umar	50	Bangalore	4
3	Jane	Jane	50	Hyderbad	4
4	Uttam	Uttam	67	Bangalore	5
5	Kim	Kim	55	Delhi	10

In [82]:
```
x_iv.columns
```

Out[82]:
```
Index(['Name', 'Domain', 'Age', 'Location', 'Exp'], dtype='object')
```

In [83]:
```
clean_data.columns
```

Out[83]:
```
Index(['Name', 'Domain', 'Age', 'Location', 'Salary', 'Exp'], dtype='object')
```

In [84]:
```
clean_data
```

Out[84]:

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Mike	34	Mumbai	5000	2
1	Teddy	Teddy	45	Bangalore	10000	3
2	Umar	Umar	50	Bangalore	15000	4
3	Jane	Jane	50	Hyderbad	20000	4
4	Uttam	Uttam	67	Bangalore	30000	5
5	Kim	Kim	55	Delhi	60000	10

In [85]:
```
y_dv = clean_data.drop(['Name', 'Domain', 'Age', 'Location','Exp'],axis=1)
```

In [86]:
```
y_dv
```

Out[86]:

	Salary
0	5000
1	10000
2	15000
3	20000
4	30000
5	60000

In [87]:
```
clean_data
```

Out[87]:

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Mike	34	Mumbai	5000	2
1	Teddy	Teddy	45	Bangalore	10000	3
2	Umar	Umar	50	Bangalore	15000	4
3	Jane	Jane	50	Hyderbad	20000	4
4	Uttam	Uttam	67	Bangalore	30000	5
5	Kim	Kim	55	Delhi	60000	10

In [88]:
```
x_iv
```

Out[88]:

	Name	Domain	Age	Location	Exp
0	Mike	Mike	34	Mumbai	2
1	Teddy	Teddy	45	Bangalore	3
2	Umar	Umar	50	Bangalore	4
3	Jane	Jane	50	Hyderbad	4
4	Uttam	Uttam	67	Bangalore	5
5	Kim	Kim	55	Delhi	10

In [89]:
```
y_dv
```

Out[89]:

	Salary
0	5000
1	10000
2	15000
3	20000
4	30000
5	60000

In [90]:
```
clean_data
```

Out[90]:

	Name	Domain	Age	Location	Salary	Exp
0	Mike	Mike	34	Mumbai	5000	2
1	Teddy	Teddy	45	Bangalore	10000	3
2	Umar	Umar	50	Bangalore	15000	4
3	Jane	Jane	50	Hyderbad	20000	4
4	Uttam	Uttam	67	Bangalore	30000	5
5	Kim	Kim	55	Delhi	60000	10

In [91]:
```
imputation = pd.get_dummies(clean_data)
```

In [92]:
```
imputation
```

Out[92]:

	Age	Salary	Exp	Name_Jane	Name_Kim	Name_Mike	Name_Teddy	Name_Umar	Nan
0	34	5000	2	False	False	True	False	False	
1	45	10000	3	False	False	False	True	False	
2	50	15000	4	False	False	False	False	True	
3	50	20000	4	True	False	False	False	False	
4	67	30000	5	False	False	False	False	False	
5	55	60000	10	False	True	False	False	False	

In [ ]: